

Section 1: Design Applications (14%) - Study Guide

Disclaimer based on your instructions: My automated web search tool primarily surfaced third-party aggregators (Medium, GitHub, etc.) rather than direct `docs.databricks.com` URLs for some of these specific exam phrasing exact matches. Therefore, as requested, I am explicitly stating this limitation. I have constructed this guide based on my knowledge of current Databricks products (like Mosaic AI Agent Framework, MLflow, and Foundation Model APIs). For each section, I have provided the specific search paths and Databricks features you should look up on `docs.databricks.com` to verify these concepts.

Terminology Breakdown

Before diving into the exam objectives, here is a breakdown of the key terminology used in this section, grounded in the Databricks ecosystem:

- **LLM (Large Language Model):** A foundational AI model trained on vast amounts of text, capable of understanding and generating human language (e.g., Llama 3, DBRX).
- **Prompt / Prompt Engineering:** The text or instructions provided to an LLM to guide its output. Prompt engineering is the practice of refining this text to elicit the most accurate and useful response.
- **JSON (JavaScript Object Notation):** A standard text-based format for representing structured data. In GenAI, forcing an LLM to output JSON is critical for integrating AI into traditional software applications.
- **RAG (Retrieval-Augmented Generation):** An architectural pattern that improves LLM responses by first retrieving relevant facts from an external knowledge base (like a company's private documents via Databricks Vector Search) and passing them to the model as context.
- **LangChain:** A popular open-source framework for building applications powered by LLMs. Databricks integrates deeply with LangChain via MLflow (`mlflow.langchain`).
- **Chain / LLM Chain:** A sequence of modular components where the output of one component becomes the input of the next (e.g., User Input -> Retriever -> Prompt -> LLM -> Output).
- **Output Parser:** A component (often in LangChain) that takes the raw text generated by an LLM and converts it into a structured format like a Python dictionary or a JSON object.
- **Retriever:** A component in a RAG system responsible for searching a database (like Databricks Vector Search) and returning documents most relevant to a user's query.
- **Databricks Model Serving / Foundation Model APIs:** The Databricks service that hosts machine learning models (including LLMs) and exposes them as secure REST APIs so applications can query them.
- **MLflow:** An open-source platform developed by Databricks for managing the end-to-end machine learning lifecycle, including tracking experiments, packaging code, and sharing models.

- **MLflow Model Signature:** A formal definition logged with a model in MLflow that specifies the exact data types and shapes the model expects for its inputs and outputs.
- **Unity Catalog:** Databricks' unified governance solution for data and AI assets. It manages permissions for tables, files, models, and tools (functions).
- **Mosaic AI Agent Framework:** Databricks' toolset designed to help developers build, deploy, and evaluate autonomous AI agents and RAG applications securely.
- **Agent / Agent Bricks:** An AI agent is a system where an LLM uses tools to achieve a goal. "Agent Bricks" refers to architectural templates (like building blocks) provided by Databricks to quickly deploy specific types of agents.
- **Multi-stage Reasoning:** A process where an AI agent breaks down a complex request into smaller steps, planning and executing multiple actions (like searching a database, then analyzing the results) before providing a final answer.
- **Databricks AI Functions:** Built-in SQL functions (e.g., `ai_extract` , `ai_summarize`) that allow users to invoke LLM capabilities directly within SQL queries on Databricks.

1. Design a prompt that elicits a specifically formatted response

Concept: In enterprise Generative AI applications, you often need the LLM to return data in a structured format (like JSON) so it can be parsed programmatically by downstream systems, rather than returning conversational text.

Databricks Context & Implementation:

- * **Prompt Instructions:** You must explicitly instruct the model in the system prompt to return a specific format (e.g., "Return ONLY a valid JSON object with the keys 'summary' and 'sentiment'. Do not include markdown formatting.>").
- * **Output Parsers:** When building LangChain applications on Databricks, you use components like `JsonOutputParser` or `PydanticOutputParser` . These inject formatting instructions into your prompt automatically and parse the resulting string into a structured Python object.
- * **Model Serving / Foundation Model APIs:** When querying Databricks Foundation Model APIs (like `databricks-meta-llama-3-70b-instruct`), you pass the prompt in the `messages` array. Some advanced models support structured output parameters directly in the API call.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: `* "Query generative AI models" (Foundation Model APIs) * "MLflow LangChain flavor" ( mlflow.langchain ) * "Prompt engineering best practices"`

2. Select model tasks to accomplish a given business requirement

Concept: Different business problems require different NLP/GenAI task types. You must match the requirement to the correct model capability.

Databricks Context & Implementation: When configuring a model in Databricks (e.g., using MLflow or Databricks SQL AI Functions), you categorize the task. Common mappings include: * **Business Requirement:** "We need to identify if customer reviews are positive or negative." -> **Task:** Text Classification / Sentiment Analysis. * **Business Requirement:** "We need a chatbot that answers questions based on our docs." -> **Task:** Question Answering / Chat. * **Business Requirement:** "We need to pull invoice numbers and dates from raw email text." -> **Task:** Information Extraction (using functions like `ai_extract()`). * **Business Requirement:** "Condense a 50-page report into a paragraph." -> **Task:** Summarization (using functions like `ai_summarize()`).

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "AI Functions on Databricks" (specifically `ai_extract`, `ai_summarize`, `ai_classify`) * "MLflow NLP tasks"

3. Select chain components for a desired model input and output

Concept: An LLM "chain" is a sequence of operations where the output of one component serves as the input to the next. You must select the right components to transform the user's raw input into the final desired output.

Databricks Context & Implementation: When building a RAG (Retrieval-Augmented Generation) chain using Databricks and MLflow (often utilizing LangChain), the standard components include: 1. **Input:** User query (string). 2. **Retriever:** Databricks Vector Search retriever (takes the string, outputs relevant Document chunks). 3. **Prompt Template:** Combines the user query and the retrieved documents into a single formatted string. 4. **LLM:** A Databricks Model Serving endpoint (takes the formatted string, outputs a generated string). 5. **Output Parser:** Parses the LLM's string into the final desired format (e.g., JSON).

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Build a RAG application with Mosaic AI Vector Search" * "Log and load LangChain models with MLflow"

4. Translate business use case goals into a description of the desired inputs and outputs for the AI pipeline

Concept: Before coding, you must define the contract of your AI pipeline. What exactly goes in, and what exactly comes out?

Databricks Context & Implementation: In Databricks, this translation is formalized using **MLflow Model Signatures**. When you log an AI model or a LangChain agent using `mlflow.langchain.log_model()`,

you must provide a `signature`. The signature explicitly defines: * **Inputs:** The expected schema of the input (e.g., a dictionary containing a `query` string and a `chat_history` list). * **Outputs:** The expected schema of the output (e.g., a string response, or a JSON object with specific fields). Defining this ensures that when the model is deployed to Databricks Model Serving, the endpoint API enforces the correct input/output schema.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "MLflow Model Signatures" * "Log a GenAI model with MLflow"

5. Define and order tools that gather knowledge or take actions for multi-stage reasoning

Concept: Multi-stage reasoning involves an AI Agent breaking down a complex problem into steps, planning which tools to use, executing them, and reasoning over the results.

Databricks Context & Implementation: This is central to the **Mosaic AI Agent Framework**. * **Tools:** In Databricks, tools are typically defined as **Unity Catalog Functions** or standard Python functions. Unity Catalog provides governed access to data and tools. * **Process:** You provide the LLM with a list of available tools (e.g., a tool to query a SQL database, a tool to search the web, a tool to fetch customer details). The LLM uses a framework (like LangChain's AgentExecutor or an OpenAI-compatible function-calling model hosted on Databricks) to decide *which* tool to call, *when* to call it, and how to combine the results to formulate the final answer.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Mosaic AI Agent Framework" * "Create tools for agents using Unity Catalog functions" * "Function calling with Databricks Foundation Model APIs"

6. Determine how and when to use Agent Bricks (Knowledge Assistant, Multiagent Supervisor, Information Extraction) to solve problems

Concept: "Agent Bricks" refer to architectural patterns or reference templates for building specific types of generative AI solutions on Databricks.

Databricks Context & Implementation: * **Knowledge Assistant:** * *When to use:* When you need a conversational chatbot that grounds its answers in your private, unstructured data (e.g., a bot answering HR policy questions). * *How:* Implemented as a RAG architecture using Databricks Vector Search, an LLM, and the Agent Framework. * **Multiagent Supervisor:** * *When to use:* When tasks are too complex for a single agent and require routing. For example, a user asks a question that might require querying a SQL database OR searching a PDF repository. * *How:* A "Supervisor" LLM analyzes the intent and routes the query to specialized sub-agents (e.g., a SQL Agent or a Document Agent), then synthesizes their responses. * **Information Extraction:** * *When to use:* When you need to convert unstructured text (emails,

PDFs, call transcripts) into structured tabular data (e.g., Delta tables). * *How*: Often implemented using Databricks SQL AI functions like `ai_extract()` which allows you to pass unstructured text and a schema (like a JSON struct), and the LLM extracts the entities into columns.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Mosaic AI Agent Framework tutorials" * "Parse and extract data with ai_extract" * "RAG reference architecture on Databricks"